Codopi Project Documentation

1. Project Title

Codopi: A Desktop-Based Compiler IDE for C, C++, CodopiLang, and Python-to-C Conversion

2. Project Overview

Codopi is a desktop-based compiler and code editor application. It is designed to work like a lightweight IDE, similar to Visual Studio Code, but focused on compiling and running C, C++, a custom language called CodopiLang, and a simple Python-to-C converter.

The main goal of this project is to create a user-friendly desktop compiler environment where users can write code, save files, open folders, compile programs, see errors, run programs, and interact with the program through a live terminal.

Codopi is not only a code editor. It also connects the editor with real compilers and a terminal system. This allows the user to write code and run it directly from the application.

3. Type of Project

Codopi is a **desktop-based compiler IDE**.

It is desktop-based because it is built using Electron. Electron allows web technologies like HTML, CSS, JavaScript, React, and TypeScript to run inside a desktop application window.

During development, Codopi may show a local URL such as:

```
http://127.0.0.1:5173
```

This URL is only used by Vite during development. It does not mean Codopi is only a website. The actual application runs inside an Electron desktop window. In the final packaged version, Codopi will load local files instead of using the development URL.

4. Main Purpose of the Project

The purpose of Codopi is to provide a desktop environment where users can:

- Write C and C++ code
- Compile C and C++ code using GCC and G++
- Run programs from inside the app
- Use live terminal input for `cin`, `scanf`, and other input functions

- See compiler errors clearly
- Open and save files
- Open folders and view project files
- Work with a custom language named CodopiLang
- Use Flex and Bison for lexical analysis and parsing in CodopiLang
- Convert a simple subset of Python code into C code
- View generated C code from Python-to-C conversion
- Use a VS Code-like interface

5. Technologies Used

5.1 Electron

Electron is used to make Codopi a desktop application.

Electron combines Chromium and Node.js. Chromium displays the user interface, while Node.js allows the app to access the computer system, files, folders, compilers, and terminal processes.

In Codopi, Electron is used for:

- Creating the desktop window
- Loading the React interface
- Handling file opening and saving
- Running compiler commands
- Communicating between the frontend and backend
- Managing the live terminal
- Controlling minimize, maximize, and close buttons

The main Electron backend file is:

```
electron/main.cjs
```

This file controls the main process of the application.

5.2 React

React is used to build the user interface of Codopi.

The interface includes:

- Code writing area
- Menu bar
- Sidebar
- Welcome screen
- Problems panel
- Output panel

- Terminal panel
- Buttons such as Run, New File, Save, Save As, and Open Folder

React makes the interface dynamic. When the user writes code, changes language, opens files, or runs code, the UI updates immediately.

The main React file is:

```
src/App.tsx
```

5.3 TypeScript

TypeScript is used with React to make the code safer and more structured.

TypeScript helps define what kind of data is passed between the frontend and Electron backend. For example, Codopi uses TypeScript types for:

- Supported languages
- Compiler results
- Problems and errors
- Open file results
- Save file results
- Folder file data

The TypeScript declaration file is:

```
src/codopi.d.ts
```

This file tells TypeScript what functions exist inside:

```
window.codopi
```

For example:

```
window.codopi.runCode()
window.codopi.openFile()
window.codopi.saveFile()
window.codopi.terminalWrite()
```

5.4 Vite

Vite is used as the frontend development server and build tool.

During development, Vite runs the React app at:

```
http://127.0.0.1:5173
```

Electron loads this URL inside the desktop window while developing.

Vite is useful because it starts quickly, updates the UI fast, and builds the final frontend files for production.

5.5 Node.js

Node.js is used inside Electron's backend process.

Node.js allows Codopi to:

- Read files
- Write files
- Open folders
- Create temporary files
- Run compiler commands
- Start and stop terminal processes
- Communicate with the operating system

Node.js modules used in Codopi include:

```
fs
path
os
child_process
```

5.6 GCC

GCC is used to compile C programs.

When the user writes C code, Codopi saves the code temporarily as a `.c` file and then runs GCC to compile it into an `.exe` file.

Example compile command:

```
gcc main.c -o main.exe
```

In Codopi, the C compiler is usually located at:

```
C:\msys64\ucrt64\bin\gcc.exe
```

5.7 G++

G++ is used to compile C++ programs.

When the user writes C++ code, Codopi saves the code temporarily as a `.cpp` file and then runs G++ to compile it into an `.exe` file.

Example compile command:

```
g++ main.cpp -o main.exe
```

In Codopi, the C++ compiler is usually located at:

```
C:\msys64\ucrt64\bin\g++.exe
```

5.8 MSYS2

MSYS2 is used to install GCC, G++, GDB, Flex, Bison, and other compiler-related tools on Windows.

Codopi uses tools from MSYS2 because Windows does not include GCC and G++ by default.

The important MSYS2 path used in Codopi is:

```
C:\msys64\ucrt64\bin
```

This path contains tools such as:

```
gcc.exe
g++.exe
flex.exe
bison.exe
```

5.9 node-pty

`node-pty` is used to create a real terminal inside Codopi.

At first, programs that needed input, such as `cin` or `scanf`, could not work properly because normal process execution waits for input but does not provide a real interactive terminal.

`node-pty` solves this problem by creating a pseudo-terminal.

Because of `node-pty`, Codopi supports:

- `cin`
- `scanf`
- `getline`
- Real typing after the program asks for input
- Interactive program execution
- Stopping the running terminal process safely
- Terminal resizing

This makes Codopi behave more like a real IDE terminal.

5.10 Flex

Flex is used for lexical analysis in CodopiLang.

A lexical analyzer reads source code and breaks it into tokens.

For example, this code:

```
let x = 10;
print x;
```

can be broken into tokens such as:

```
LET
IDENTIFIER
ASSIGN
NUMBER
SEMICOLON
PRINT
IDENTIFIER
SEMICOLON
```

Flex helps CodopiLang detect lexical errors, such as unsupported characters or invalid symbols.

The Flex file usually has this type of extension:

```
.l
```

Example:

```
codopi_lexer.l
```

5.11 Bison

Bison is used for parsing in CodopiLang.

After Flex creates tokens, Bison checks whether those tokens follow the grammar rules of the language.

For example, Bison checks if this structure is valid:

```
let x = 10;
while (x < 20) {
    print x;
    x = x + 1;
}
```

Bison helps detect syntax errors, such as missing semicolons, wrong expressions, or invalid statement order.

The Bison file usually has this type of extension:

```
.y
```

Example:

```
codopi_parser.y
```

5.12 CodopiLang

CodopiLang is the custom language part of this project.

It is a small programming language created for learning compiler design. It uses Flex and Bison for lexical analysis and parsing.

CodopiLang supports simple features such as:

- Variable declaration
- Assignment
- Print statement
- While loop
- Basic arithmetic
- Basic comparison

Example CodopiLang code:

```
let x = 10;
print x;

while (x < 13) {
    print x;
    x = x + 1;
}
```

CodopiLang follows this pipeline:

```
CodopiLang source code
→ Flex lexer
→ Bison parser
→ C code generator
→ GCC backend
→ Executable program
→ Output
```

This makes CodopiLang a real basic compiler pipeline.

5.13 Python-to-C Converter

Codopi also includes a simple Python-to-C converter.

This converter does not support full Python. Full Python is very large and includes many advanced features like lists, dictionaries, classes, imports, exceptions, dynamic typing, and many built-in functions.

Instead, Codopi supports a useful student-level subset of Python.

Supported Python-to-C features include:

- Integer variables
- `int(input("message"))`
- `print()`
- Multiple print arguments like `print("Sum:", a + b)`
- Arithmetic operations
- If-else statements
- While loops
- Comparison operators

Example Python code:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print("Sum:", a + b)
print("Difference:", a - b)
print("Product:", a * b)
print("Division:", a / b)
```

Codopi converts this into C code similar to:

```
#include <stdio.h>

int main() {
int a;
printf("Enter first number: ");
fflush(stdout);
scanf("%d", &a);
int b;
printf("Enter second number: ");
fflush(stdout);
scanf("%d", &b);
printf("Sum: %d\n", a + b);
printf("Difference: %d\n", a - b);
printf("Product: %d\n", a * b);
printf("Division: %d\n", a / b);
return 0;
}
```

After conversion, Codopi compiles the generated C code using GCC and runs it in the live terminal.

6. Main Files of the Project

6.1 electron/main.cjs

This is the Electron main process file.

It handles backend operations such as:

- Creating the app window
- Running GCC and G++
- Running the CodopiLang compiler
- Running the Python-to-C converter
- Managing the live terminal
- Opening files
- Saving files

- Opening folders
- Parsing compiler errors
- Sending terminal output to the frontend
- Stopping terminal processes safely

Important modules used in this file:

```
const { app, BrowserWindow, ipcMain, dialog } = require("electron");
const path = require("path");
const fs = require("fs");
const os = require("os");
const { spawn } = require("child_process");
const pty = require("node-pty");
```

6.2 electron/preload.cjs

This file safely connects the frontend React app with the Electron backend.

Because Codopi uses `contextIsolation: true`, the frontend cannot directly access Node.js or Electron APIs. This is safer.

The preload file exposes safe functions through:

```
contextBridge.exposeInMainWorld("codopi", {
    ...
});
```

This allows the frontend to use functions like:

```
window.codopi.runCode()
window.codopi.openFile()
window.codopi.saveFile()
window.codopi.terminalWrite()
```

6.3 src/App.tsx

This is the main frontend file.

It controls the user interface and app behavior.

It handles:

- Code editor area

- Language selection
- Run button
- New File
- Open File
- Save File
- Save As
- Open Folder
- Problems panel
- Output panel
- Terminal panel
- Live terminal input
- Resizable panels
- Welcome screen
- UI layout

6.4 src/App.css

This file controls the visual design of Codopi.

It includes styles for:

- Dark theme
- Sidebar
- Top menu
- Editor area
- Output panel
- Problems panel
- Terminal panel
- Buttons
- Welcome screen
- Logo placement
- Resizable sections

6.5 src/codopi.d.ts

This TypeScript declaration file defines Codopi's frontend API.

It tells TypeScript what functions and data types exist in:

```
window.codopi
```

This prevents TypeScript errors and makes the app easier to maintain.

6.6 compiler-lab/codopi_lexer.l

This file contains the Flex lexer rules for CodopiLang.

It detects tokens such as:

- Numbers
- Identifiers
- Keywords
- Operators
- Symbols
- Invalid characters

6.7 compiler-lab/codopi_parser.y

This file contains the Bison grammar rules for CodopiLang.

It checks whether the token sequence follows valid language grammar.

It also helps generate C code from CodopiLang.

6.8 compiler-lab/codopi_lang_compiler.exe

This is the compiled custom compiler for CodopiLang.

Codopi runs this executable when the user selects CodopiLang and compiles `.copi` code.

7. How the C and C++ Compilation Works

When the user writes C or C++ code and clicks Run, Codopi follows these steps:

1. Reads the code from the editor
2. Checks the selected language
3. Creates a temporary folder
4. Saves the code as a temporary `.c` or `.cpp` file
5. Selects GCC for C or G++ for C++
6. Runs the compiler command
7. Captures compiler errors
8. Shows errors in the Problems panel
9. If compilation succeeds, starts the executable in the live terminal
10. Allows the user to type input directly into the terminal

For C:

```
gcc source.c -o program.exe
```

For C++:

```
g++ source.cpp -o program.exe
```

8. How Error Detection Works

Codopi captures compiler output from GCC, G++, CodopiLang, and Python-to-C conversion.

For C and C++, GCC/G++ errors usually look like:

```
main.cpp:5:10: error: expected ';' before '}'
```

Codopi parses this output and extracts:

- File name
- Line number
- Column number
- Error type
- Error message

Then it shows the result in the Problems panel.

Codopi can show:

- Compilation errors
- Warnings
- Notes
- Lexical errors
- Syntax errors
- Semantic errors

9. Lexical Error, Syntax Error, and Semantic Error

9.1 Lexical Error

A lexical error happens when the code contains an invalid character or symbol.

Example:

```
let x = 10 @;
```

The `@` symbol may not be allowed in CodopiLang, so the lexer reports a lexical error.

9.2 Syntax Error

A syntax error happens when the structure of the code is wrong.

Example:

```
let x = ;
```

This is invalid because a value is missing after `=`.

9.3 Semantic Error

A semantic error happens when the grammar may be correct, but the meaning is wrong.

Example:

```
print y;
```

If `y` was never declared, this can be considered a semantic error.

10. How the Live Terminal Works

The live terminal works using `node-pty`.

Normal process running with `child_process.spawn()` is useful for compiling, but it is not enough for fully interactive programs.

For example, this C++ code needs real input:

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    if (num % 2 == 0)
        cout << "Even";
    else
        cout << "Odd";
```

```
    return 0;
}
```

Without a live terminal, the program may wait forever or not allow typing.

With `node-pty`, Codopi starts the compiled `.exe` inside a real terminal session. The user can click inside the terminal, type input, and press Enter.

The terminal supports:

- Real-time output
- Real-time input
- `cin`
- `scanf`
- `getline`
- Process exit messages
- Stopping the running process
- Resizing terminal dimensions

11. How File and Folder Features Work

Codopi supports file and folder features using Electron's dialog API and Node.js file system module.

New File

Creates a new blank editor file.

Open File

Uses Electron dialog to select a file from the computer and loads it into the editor.

Save File

Saves the current code into the current file path. If the file does not already have a path, Codopi asks the user where to save it.

Save As

Always asks the user to choose a location and file name.

Open Folder

Allows the user to open a folder and view supported files from that folder.

Supported file types include:

```
.c
.cpp
.h
.hpp
.copi
.py
.txt
```

12. User Interface Design

Codopi's interface is designed to be similar to a modern IDE.

Main UI sections include:

- Top title bar
- Menu bar
- Sidebar
- Code editor
- Output panel
- Terminal panel
- Problems panel
- Welcome screen
- Logo section
- Run button
- Language selector

The bottom panels are ordered as:

```
Output → Terminal → Problems
```

The panels can be resized so the user can make the needed section bigger or smaller.

The interface uses a dark theme to make it comfortable for coding.

13. Why Codopi Uses Electron Instead of Only a Website

A normal website cannot directly access the computer's file system, run GCC, start a terminal, or compile code locally.

Codopi needs these features:

- Run local compilers
- Access files and folders
- Start `.exe` programs

- Use a real terminal
- Communicate with MSYS2 tools
- Save files directly on the computer

That is why Electron is used.

Electron allows Codopi to behave like a desktop app while still using web technologies for the interface.

14. Why Codopi Uses React

React is used because it makes it easier to build a dynamic and interactive interface.

Codopi needs many changing UI parts, such as:

- Current code
- Current file name
- Current folder
- Compiler output
- Terminal output
- Error list
- Selected language
- Active panel
- Resizable sections

React handles these changes efficiently.

15. Why Codopi Uses TypeScript

TypeScript helps make the project more organized and safer.

It helps define data structures clearly, such as:

```
CodopiProblem
CodopiRunResult
CodopiOpenFileResult
CodopiSaveFileResult
CodopiLanguage
```

This reduces mistakes while passing data between Electron and React.

16. Why Codopi Uses GCC and G++

GCC and G++ are real compilers.

Codopi does not fake C or C++ compilation. It uses actual compiler tools.

This means the output and errors are real.

GCC is used for C.

G++ is used for C++.

17. Why Codopi Uses Flex and Bison

Flex and Bison are used because they are standard tools for compiler construction.

Flex handles lexical analysis.

Bison handles parsing.

Together, they allow CodopiLang to work like a real small programming language.

This is important because the project is not only a code editor. It also demonstrates real compiler design concepts.

18. Why Codopi Uses Python-to-C Conversion

The Python-to-C feature adds another compiler-related function.

It shows how a high-level language can be converted into C code.

This helps demonstrate:

- Source-to-source translation
- Code generation
- Syntax checking
- Input/output conversion
- Compilation through GCC

The Python-to-C converter currently supports a limited subset of Python, but it is useful for basic programs.

19. Project Workflow

The general workflow of Codopi is:

```
User writes code
→ User selects language
→ User clicks Run
→ Codopi saves code temporarily
→ Codopi chooses the correct compiler/converter
→ Code is compiled or converted
```

```
→ Errors are shown if found
→ If successful, program runs in live terminal
→ User can type input
→ Output is shown
```

20. Supported Languages

Codopi currently supports:

C

Compiled using GCC.

C++

Compiled using G++.

CodopiLang

Processed using Flex, Bison, C code generation, and GCC.

Python-to-C

Converted into C code, then compiled using GCC.

21. Current Limitations

Codopi is still a developing project.

Current limitations include:

- Python-to-C does not support full Python
- CodopiLang is still a small custom language
- The editor is not yet as advanced as VS Code
- Advanced debugging with breakpoints is not fully implemented
- Some advanced C/C++ project systems are not yet supported
- Full IntelliSense is not yet implemented
- Full semantic suggestions are still limited

22. Future Improvements

Possible future improvements include:

- Add syntax highlighting using Monaco Editor
- Add IntelliSense-style suggestions

- Add debugger support using GDB
- Add breakpoints
- Add project build system support
- Add custom themes
- Add autocomplete
- Add better CodopiLang grammar
- Add functions in CodopiLang
- Add arrays in CodopiLang
- Add string support in Python-to-C
- Add better error underlining
- Add code formatting
- Add file tabs
- Add integrated settings page
- Add final Windows installer
- Add custom Codopi app icon
- Add project templates

23. Professional Project Description

Codopi is a desktop-based compiler IDE developed using Electron, React, TypeScript, and Node.js. It provides a VS Code-like coding environment where users can write, save, open, compile, and run programs. The application supports C and C++ compilation using GCC and G++, a custom compiler pipeline for CodopiLang using Flex and Bison, and a simple Python-to-C transpiler that converts supported Python syntax into C code before compiling it with GCC.

The project includes a live terminal powered by node-pty, allowing interactive input and output for programs that use `cin`, `scanf`, `getline`, and similar input methods. It also includes an output panel, terminal panel, problems panel, file management features, folder opening, and compiler error parsing.

Codopi demonstrates practical knowledge of desktop application development, compiler design, process management, terminal integration, and user interface design.

24. Final Summary

Codopi is a desktop-based compiler environment that combines a code editor, compiler system, terminal, file manager, and custom language support in one application.

It uses Electron to run as a desktop app, React and TypeScript for the interface, Node.js for backend operations, GCC and G++ for C/C++ compilation, Flex and Bison for CodopiLang, and node-pty for live terminal support.

This project is important because it shows how a real desktop IDE can be built from scratch using modern technologies and real compiler tools.